Technical Design Document — Mistridge Estate direct booking and enquiry router

GitHub: https://github.com/keshav183/mistridge-estate *(replace if your repo name differs)*
Live app: *add URL after deploying (see deployment.md)* **Client brief:** Company A, coffee-estate resort, Chikkamagaluru (76 keys). The UI uses a placeholder brand, "Mistridge Estate".

1. What was built

A booking website where guests choose dates, see real prices with GST, and pay (test mode) in three steps, with no account needed; a staff admin for daily rates, website allotment, blackout dates and promo codes; one enquiry inbox for web, event-form, forwarded-email and WhatsApp messages (the last two simulated) with an AI router that classifies, extracts, assigns an owner and drafts a reply; and a daily CSV booking export. The core slice from the brief (room pages with availability and price, booking with test payment, rate and allotment admin, enquiry inbox with AI routing, guest and reservations roles) works end to end on seeded sample data. The structured event form, GM alerts, promo codes, export and booking lookup are included as depth. Guests book without an account; sign-in is for staff (reservations, sales, GM).

Why it matters (from the brief): about ₹1.05 crore a year at stake: ₹33.3 lakh from moving 15% of agency room nights to direct, plus ₹72 lakh from event enquiries lost to slow replies. The design maps to the three success measures: direct share 12% → 20%, median first reply under 30 minutes, zero website overbookings. The inbox shows the median first-reply time live.

2. Assumptions (labelled)

#	Assumption	Where it shows up
A1	The resort's brand is anonymised in the brief; "Mistridge Estate" is a placeholder in one constant.	UI, drafts
A2	Four room types split the 76 keys: Valley Deluxe 40, Canopy Suite 24 (64 rooms); Planter's Villa 8, Ridge Pool Villa 4 (12 villas). Base rates ₹6,200–₹19,500 with seasonal and weekend factors, giving a blended rate near the brief's ₹9,500.	Seed
A3	GST on accommodation: 5% (no ITC) when the value charged per room per night is ₹7,500 or less, 18% above, effective 22 Sep 2025. Sources differ on whether the test uses the declared or the charged tariff; I use the charged nightly value after promo discount. Breakfast bundled into one tariff is taxed at the room rate. Rates are in one file (`lib/pricing.js`) for a one-line change. A chartered accountant must confirm before launch.	Pricing
A4	Rate plans: Flexible (free cancellation to 3 days before arrival), With	Seed

#	Assumption	Where it shows up
	breakfast (+₹1,200 a night per room), Advance saver (10% off, non-refundable).	
A5	Website allotment is a number of units per room type per night set by the admin; it is never more than physical units and never below what is sold.	Admin, engine
A6	Online stays are capped at 14 nights and 400 days ahead. Longer stays go through enquiry.	Validation
A7	Event enquiries carry no price in drafts: event pricing is not in the rate table.	Drafts
A8	"Working hours" and cancellation/refund execution are outside this build; cancelling releases the room, refunds are handled offline.	Admin
A9	The daily export contains bookings created (and since cancelled) on the chosen day, in a flat CSV; the real reservation system's import format was not specified.	Export
A10	Photos are illustrated vector stand-ins generated by a script; real photography replaces them without code changes.	Frontend

3. Architecture and why

Stack: MERN. React 18 (Vite) single-page app; Node/Express API; MongoDB with Mongoose; JWT in an httpOnly cookie. **AI: Google Gemini** (REST, optional) on top of a built-in rules engine. **MongoDB is optional locally:** with no `MONGODB_URI` the API starts an embedded MongoDB (`mongodb-memory-server`) that keeps its files in `backend/.data` and seeds itself, so `npm run dev` works on a clean machine; setting `MONGODB_URI` switches to a real MongoDB or Atlas (used in production).

```
Browser (React SPA, code-split admin)
   │  JSON over HTTPS, same origin, httpOnly SameSite=Lax session
cookie
Express API ── helmet, rate limits, zod validation, role middleware
   ├─ routes/public     rooms, availability, price, bookings, lookup
   ├─ routes/admin      inventory bulk edit, promos, bookings, CSV
export, alerts
   ├─ routes/enquiries  web form, ingest (email/WhatsApp), inbox, re-
route, reply
   ├─ lib/              dates, pricing+GST, inventory engine,
payments(mock), csv  (pure, unit-tested)
   ├─ services/         availability, booking orchestration, enquiry
router
   └─ ai/               rules engine, Gemini wrapper, classifier,
draft guard, owner assignment
MongoDB: users, roomtypes, rateplans, inventories, promocodes,
bookings, enquiries, notifications, counters
```

Key decisions

- **One service, one origin.** Express serves the built React app. The session cookie is first-party, there is no CORS surface, and there is one thing to deploy for an ₹8 lakh, 7-week project.
- **Cookie sessions, not tokens in localStorage.** JavaScript cannot read the session, which limits damage from XSS. SameSite=Lax plus JSON-only endpoints mitigates CSRF.
- **MongoDB (optional to install).** Documents fit bookings (a price snapshot belongs inside the booking) and enquiries (AI output, history and replies are naturally nested). The one place that needs relational care, stock, is handled by a single-document atomic update (section 5).
- **Pure logic in `lib/`.** Pricing, dates, the inventory engine and the CSV writer have no database or HTTP in them, so the important rules are unit-tested without infrastructure.
- **Speed on slow networks.** No web fonts (system stacks), vector photos of 4–10 KB each, route-level code splitting (guest bundle about 67 KB gzipped; staff screens load only for staff), aspect-ratio boxes so nothing jumps, lazy-loaded images, long-cache hashed assets, gzip.

4. Data model

Collection	Key fields	Notes
`User`	name, email (unique), passwordHash, role: reservations / sales / gm	Staff only; guests book without an account. bcrypt cost 11.
`RoomType`	slug, name, kind (room/villa), units, maxAdults, maxGuests, photos[], amenities[]	Physical inventory.
`RatePlan`	code, adjustPct, addPerNight, refundable, cancellation	Price = base × (1 + adjust) + add-on.
`Inventory`	roomType, date (YYYY-MM-DD), rate, allotment, sold, blackout	**Unique (roomType, date).** One row per room type per night. The only stock the website can sell.
`PromoCode`	code, percent/flat, value, maxDiscount, minNights, validTo, maxUses, used, active	Redeemed with an atomic counter.
`Booking`	code, idempotencyKey (unique), roomType, plan, dates, guest{name,email,phone}, pricing (full snapshot), promo, payment{status, last4}, consentAt, status, createdDay	Price snapshot means later rate edits never change a confirmed booking. No card number or CVC is stored.
`Enqui`	ref (ENQ-4410…), channel,	Lane is separate from ai.category so a human

Collection	Key fields	Notes
`ry`	message, form, lane, owner, status, gmAlert, ai{category, confidence, source, summary, extracted, warnings, draft}, replies[], history[], firstReplyAt, consentAt	re-route keeps the AI's original reading for audit.
`Notification`	user, enquiry, text, readAt	GM complaint alerts (the WhatsApp/SMS send is simulated and logged).

Dates are plain `YYYY-MM-DD` strings in the property's calendar (IST). That avoids timezone drift and makes "nights between two dates" a simple range query.

5. The hard part: allotment control, so the site never overbooks

Rule: the website may sell at most `allotment − sold` units of a room type on any night, and never on a blacked-out night.

Mechanism: a hold is one atomic conditional update per night:

```
updateOne(
  { roomType, date, blackout: false, $expr: { $lte: [ { $add:
['$sold', qty] }, '$allotment' ] } },
  { $inc: { sold: qty } })           // success only if modifiedCount
=== 1
```

There is no read-then-write. If two guests race for the last unit, MongoDB serialises the updates on that document and exactly one succeeds. A multi-night stay holds night by night; if any night fails, the nights already held are released, and the guest sees "the last room was just taken".

Booking sequence (`services/booking.js`): validate and price on the server (never trusting client prices) → idempotency check → hold every night → claim promo atomically → charge (mock) → persist the booking. A failure at any step after the hold releases the nights and the promo. A required `Idempotency-Key` header, unique-indexed on the booking, means a double click or retry returns the same booking and never a second one.

Admin safety: allotment cannot exceed physical units, and a bulk change never lowers allotment below what is already sold (those nights are skipped and reported). Cancelling releases the nights.

Tests (`test/inventory.test.js`): 50 simultaneous bookings for 3 units yield exactly 3 successes; overlapping multi-night stays under concurrency never exceed allotment on any night; partial failure rolls back; blackout and missing nights cannot be held. The in-memory store used in the tests mirrors Mongo's atomic conditional update semantics.

6. Pricing and GST

`quoteStay()` computes, per room per night: plan price − promo discount spread proportionally before tax − GST slab chosen from what the guest pays that night (5% up to ₹7,500, otherwise 18%). Totals are shown as subtotal, discount, GST by rate and total, so a stay that crosses the threshold (a peak weekend) shows both 5% and 18% lines. Promo rules (minimum nights, validity, caps, redemption limit) are in `evaluatePromo()`; tests cover slab boundaries (₹7,500 and ₹7,501), breakfast add-ons, promo moving a night into the lower slab, multiple rooms and each promo rejection.

7. The AI part: classify, extract, assign, draft, and what happens when it is wrong

```
message → rules engine (always) ──┐
        └→ Gemini (if key set) ──────→ validate (zod) → cross-check → confidence
confidence ≥ 0.6 ?  lane = category  :  lane = Unsorted (GM)
lane → owner = least-loaded person in the lane's role
(room→reservations, wedding/offsite→sales, complaint/unsorted→GM)
room + dates → quote from the rate table → draft (token {{QUOTE}} filled by the server)
complaint (or any complaint signal) → GM notification
```

- **Classification and extraction:** category (room, wedding, offsite, complaint), dates, month hints, guests, adults/children, nights and budget (₹, lakh, crore). Indian formats are handled ("14-16 Nov", "₹3.5 lakh", "team of 45"). The rules engine alone passes the three sample enquiries (ENQ-4410 Wedding, ENQ-4411 Room, ENQ-4412 Corporate offsite) in tests.
- **Gemini layer:** one REST call with JSON output mode, thinking off for speed and a 12 s timeout. the enquiry is passed as delimited data with an instruction not to follow text inside it. Output must match a strict schema; extracted dates are re-validated and discarded if invalid or in the past.
- **When it is wrong or unsure**
  - Confidence below 0.6, or "other", goes to the **Unsorted lane that the GM owns**, with a plain-language warning in the UI.
  - If Gemini and the rules engine disagree and the rules engine is confident, confidence is capped below the threshold, so disagreement forces human review.
  - Malformed or failed model output, or a timeout (12 s), falls back to the rules engine; the enquiry is never lost or delayed.
  - **Staff re-route in one click** (lane control in the detail pane); the draft is regenerated for the new lane, the change is logged in the enquiry history.
  - **Drafts are never sent automatically.** A person reads, edits and presses Send (simulated in this build).
  - **Prices in drafts come from the rate table.** The model may write prose and a `{{QUOTE}}` token only; the server fills it. A draft containing a currency amount

is discarded and replaced by a template. Event and complaint drafts contain no prices.
  - **Complaints always alert the GM**, including weak signals that are too uncertain to leave the Unsorted lane.
- **Why a hybrid:** rules are free, instant, explainable and work without a key; the model adds nuance and drafting. Disagreement between them is a useful uncertainty signal.

8. UI and design decisions

Direction: quiet, photo-led, minimal, and one theme everywhere: black ink on white, three greys and a single accent, deep canopy green, used for actions, selected states and the sidebar. No second colour: attention is shown with weight, borders and solid ink badges, and errors with a bold "!" message and a heavier field border, never red. Left-aligned serif headlines against a plain sans for forms; square-ish 2–6 px corners; no shadows. Nothing on screen lacks a purpose: no carousels, no decorative badges. The photography stand-ins use only tones of the same green and grey, so the whole site stays inside one palette.

Screen	Decision and why
Home	Full-bleed ridge photograph with one line of copy; the date/guest search bar sits on the image edge so the first action is obvious. Results below are large photo cards with total price including GST, so the guest never meets a surprise at checkout.
Room page	Gallery, facts, and a sticky booking panel. The calendar shows a nightly price or "2 left" on each day, crosses out unavailable nights, and only lets a sold-out day serve as check-out. Rate plans show the **total** per plan.
Booking (3 steps)	Stay → Guest → Payment, as the brief allows at most three. A persistent summary shows the live total with GST lines. Promo and test-card hints are inline. The consent notice and the "passport details are taken at check-in" note are on the guest step.
Confirmation / lookup	Large reference, full price breakdown. Lookup needs reference **and** email. There is no print or PDF export on the guest or staff side.
Enquiry page + side tab	A fixed "Enquiry" tab on the right edge of every guest page (and an "Enquiry" link in the header) opens a plain-language form: message, name, contact, consent. The message goes to the AI router, which classifies it, extracts dates and guests, assigns an owner and drafts a reply; the guest only sees a reference number.
Events form	Structured: type, guests, flexible month or exact dates, nights, rooms, budget in lakh. This data fixes the category with full confidence and feeds the draft.
Rates & allotment	A 14-day grid per room type showing rate and sold/allotment; full nights tinted, blackouts hatched; click a cell to prefill a bulk-edit form with weekday chips.
Enquiry	Lane tabs with counts (Unsorted highlighted when non-empty), list, and detail

Screen	Decision and why
inbox	pane: original message, an “AI reading” card (category, confidence bar, extracted fields, warnings), one-click lane control, owner select, editable draft, history. On a phone the list and detail are separate views.

States: every data screen has a skeleton while loading, an empty state that says what to do, and an error state with Retry. Forms show field-level errors, server validation errors map to the same fields, and the offline case has its own message. Focus rings, labels, `aria-live` price updates and `prefers-reduced-motion` are respected. Layouts are responsive down to a phone (calendar collapses to one month, tables scroll inside their own container).

9. Security, privacy and compliance

- **No real secrets in the repo.** `.env` is ignored. Every secret (`JWT_SECRET`, `GEMINI_API_KEY`, `INGEST_KEY`, the demo password) defaults to the demo value `hello123` so the project runs with no setup; `GEMINI_API_KEY=hello123` means “no real key” and the rules engine is used. The app warns on start in production if `JWT_SECRET` is still `hello123`: change all of them before any real use.
- **Validated inputs.** Every endpoint validates with zod (types, ranges, formats, unknown fields stripped). Mongo operators cannot be injected because typed fields reject objects. Regexes built from user text are escaped.
- **Access control.** Roles enforced on the server (`requireRole`), not just hidden in the UI: staff-only admin and inbox, rates/promos/bookings limited to reservations and GM. Public sign-up cannot create staff. Booking lookup requires code and email. Guest views omit payment references.
- **Abuse and transport.** helmet with CSP, rate limits on login, booking, lookup and public enquiries, 50 KB body limit, Secure/httpOnly/SameSite cookies in production, bcrypt, constant-time compare on the ingest key.
- **Payments.** Mock gateway; card numbers and CVC are never stored or logged, only the last four digits.
- **CSV export** neutralises spreadsheet formula injection.
- **DPDP Act 2023.** Explicit consent checkbox on booking and event forms (timestamp stored); purpose-limited fields; passport/Form C data deliberately not collected online; logs carry no request bodies. Not built: data-erasure request flow and a retention job (see section 11).

10. Testing

`npm test` runs 29 unit tests with Node’s built-in runner and no database: the concurrency and rollback tests above; GST slab boundaries, plans, promos (including that the applied discount equals the promo amount exactly) and multi-room totals; date validation (past, impossible dates, horizon, IST rollover); mock-gateway success, declines and bad input; CSV quoting and injection; the three sample enquiries, vague and complaint messages, Indian date and budget

formats; Gemini output valid, malformed, failing, contradicting and with bad dates (using an injected fake model); the price detector and draft guard; and owner assignment.

`npm run smoke` is an end-to-end script against a running API and a real MongoDB. It checks: 25 simultaneous bookings for one remaining unit produce exactly one booking and stock never goes negative; a declined card releases the hold; the same Idempotency-Key returns the same booking; promo cap and expiry; past dates and missing consent rejected; staff-only routes return 401/403 as designed; allotment above physical units refused; blacked-out nights cannot be booked; the CSV export downloads; and the three sample enquiries plus a complaint and a vague message land in the right lanes. All 22 checks passed against MongoDB 8.3 during development.

Honest gaps: there is no automated browser test (I checked the main screens at desktop and phone width by hand), the Gemini call itself was exercised only through a fake model in tests because no API key was available while building, and the embedded MongoDB was verified with a system binary during development, so the first run on a new machine also depends on the one-time binary download working on that network.

11. What I cut, and what I would do next

Cut (deliberately): guest accounts (guests book with an email and find the booking by reference), real payment gateway and webhooks; real email/WhatsApp delivery and inbound parsing (the ingest endpoint is the seam where they plug in); refunds and guest self-cancellation; multi-room-type baskets; calendar-based event availability and event pricing; the reservation-system integration beyond the CSV; automated GM escalation if a complaint is unread; photo CMS.

Next: (1) run the smoke checks in CI and add a browser test of the booking flow; (2) real gateway (Razorpay) with webhook-confirmed payment and a short stock hold with expiry; (3) WhatsApp Business and email-forwarding adapters; (4) first-reply SLA timer and breach alerts by working hours; (5) DPDP data-erasure and retention jobs, plus an audit log of staff views of guest data; (6) price/allotment change audit trail and rate-plan restrictions (minimum stay); (7) event proposals with their own pricing; (8) real photography as AVIF/WebP with `srcset`; (9) measuring direct share and reply time on a dashboard against the 20% and 30-minute targets.

Known limits: one active physical inventory per room type (no room numbers); the rules engine is English-only (Hindi/Kannada messages go to Unsorted by design); GST needs professional sign-off (A3).